CELESTIAL MECHANICS × AI · 2026

From the next token to the next theorem



Bowen Liu

Bowen Liu 刘博文

Periodic N-body solutions, elliptic relative equilibria and linear stability.

2013–2018	**Nankai University** · Chern Institute · Math PhD
2018–2020	**University of Augsburg** · Postdoc
2020–2022	**Shanghai Jiao Tong** · Wu Wen-Tsun Asst. Prof.
2022–2024	**Huawei** · Principal Engineer
2024–now	**US e-commerce** · Applied Researcher

CELESTIAL MECHANICS RESEARCH

Periodic N-body: existence & stability

Restricted four-body elliptic equilibria

Linear instability of elliptic rhombi

Homographic Lennard–Jones solutions

> Symplectic indices · Variational methods

Overview: from token to theorem

Click a section card to jump; click a chip for a key page.

00 · OPENING & TOKEN

Token

Ten integers · vocabulary · billing

p. 4–7

Bio · 10 integers · Vocabulary · Billing

01 · MODEL

One function

Input → probability of next token

p. 8

Distribution

02 · TRAINING & PROBLEM SOLVING

Learning from reward

RL loop · GRPO · math and code

p. 9–15

RL loop · GRPO · Stateless

03 · SYSTEMS & ENGINEERING

Put the model in a system

Context · Harness · Skill · long runs

p. 16–23

Harness · System map · Skill

04 · AI × MATH

From tools to systems

Acceleration · proof graph · theorem cracking

p. 24–31

DAG/MCTS · 722 papers · NS · Cracking system

05 · CONCLUSION

From token to theorem

Probability · verification · direction

p. 32

Close

What the model sees: ten integers

EN · ONE SENTENCE

Celestial mechanics is the poetry of orbits.

Cel	estial	_mechanics	_is	_the	_poetry	_of	_or	bits	.
69126	90010	52347	382	290	38153	328	503	24043	13

46 characters → **10** tokens → 10 integers · _ marks a leading space

CHINESE · FOUNDATIONS OF CELESTIAL MECHANICS

本书系统地介绍了天体力学

本	书	系统	地	介绍	了	天	体	力	学
4087	20121	31936	5557	37887	4531	867	7074	11343	8590

12 characters → **10** tokens → 10 integers · two-character words often map to one token

01

Where do these integers come from?

02

How does it predict the next one?

03

How do we put it to work?

Tokenization is table lookup

```
Three-body problem is a classic problem in celestial mechanics.
```

Three	-body	problem	_is	_a	_classic	_problem	_in	_celestial	_mechanics	.
25843	19146	4792	382	261	13686	4792	306	134437	52347	13

_ marks a leading space · Celestial → Cel + estial

≈5
characters / English word
including the space, typical

≈4
characters / token
English rule of thumb

≈1.3
tokens / English word
≈ 5 ÷ 4

50k–200k
typical vocabulary size
50k – 200k

WHY Characters: little information, 4–5× longer → Words: vocabulary explodes (millions) → **Subwords: the compromise**

> Subwords are an engineering compromise between **sequence length** and **vocabulary size**.

Tokens: the unit of model and bill

The model computes in tokens; the bill does too.



≈ 0.3 token
per English character

≈ 0.6 token
per Chinese character

> A token is the smallest unit of both the model and the bill.

One call · off-peak
100k input + 2k output

¥0.01 cache hit

¥0.11 cache miss

> Same context, cache hit vs miss: an order of magnitude.

The vocabulary: a fixed lookup table

ID	TOKEN	
290	_the	
2392	_book	
2500	This	
18864	_fundamental	
23753	_concepts	
52347	_mechanics	
134437	_celestial	← query
199999	<|endoftext|>	special

8 rows shown — 200,019 rows in total

200,019

o200k · GPT-4o generation

- **0**
- **50k** GPT-2
- **100k** GPT-4 gen
- **128k** Llama 3
- **200k** GPT-4o gen

200,019 × 4,096 ≈ **820M parameters**

```
Vocabulary row 134437 → [0.12, −0.44, 0.08, …] · 4,096 dims
```

ID is the **row number**; the vector is the **representation**.

What the model does: input → probability distribution

Input tokens

From | the | next | token | to | the

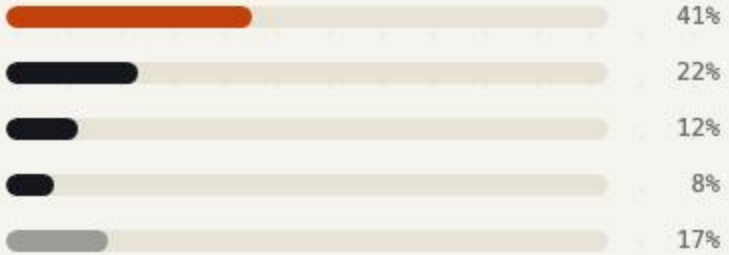
Each token is already a 4,096-dim vector looked up by ID

Model

Transformer

N layers · self-attention

softmax(z)



$$z = f_\theta(x_{1:t}) \in \mathbb{R}^{|V|} \qquad p_\theta(y \mid x_{1:t}) = \frac{\exp(z_y)}{\sum_{v \in V} \exp(z_v)}$$

$|V| = 200{,}019$

$\Sigma_y\, p_\theta(y \mid x_{1:t}) = 1$

> A scorer: it gives every token in the vocabulary a **score**, then turns scores into **probabilities**.

Training: pretraining and post-training

STAGE 01

Pretraining

Large-scale text, code and other data

Learn language, knowledge and patterns

→

STAGE 02

Post-training

SFT · SUPERVISED FINE-TUNING

Learn by example

Shape responses and task behavior.

RL · OUR FOCUS

Learn from reward

Try, evaluate, improve the policy.

> Our focus is **RL**: improving **problem-solving** through reward.

RL: actions, outcomes and feedback

Act in an environment; use outcome feedback to improve action choices.

Observe environment → Take action → Outcome → Feedback / reward → Improve policy ↺

ENVIRONMENT

Produce an outcome

Actions affect the environment; new observations return.

FEEDBACK

Evaluate the result

Reward expresses progress or success.

POLICY

Improve action choices

Use experience to seek higher long-run return.

> RL uses **outcome feedback** to learn **how to choose actions**.

Games: learn and improve in an environment

Clear rules, measurable outcomes, repeatable attempts.

2015
Google DeepMind
Atari · DQN
Pixels → actions
Reward: game score

2016
Google DeepMind
AlphaGo
Self-play + search
Reward: winning

2019
OpenAI
OpenAI Five
Dota 2 teamwork
Reward: match result

2019
Google DeepMind
AlphaStar
StarCraft II strategy
Reward: match result

TRY

Different actions

Explore timing and direction.

OBSERVE

The score

Miss the ball, or clear more bricks.

LEARN

A better policy

Learn actions with delayed benefit.

> A game engine supplies both environment and feedback; strategy grows from repeated attempts.

GRPO: objective and group advantage

Compare a group of answers to one question; no critic.

OBJECTIVE

$$J_{\mathrm{GRPO}}\left(\theta\right) = E\left[\frac{1}{G}\sum_{i}\frac{1}{|o_i|}\sum_{t}L_{i,t}\right]$$

$$L_{i,t} = \min\left[\rho_{i,t}\left(\theta\right)\hat{A}_{i,t}, \mathrm{clip}\left(\rho_{i,t}\left(\theta\right), 1-\varepsilon, 1+\varepsilon\right)\hat{A}_{i,t}\right] - \beta D_{\mathrm{KL}}$$

GROUP ADVANTAGE

$$\hat{A}_i = \frac{r_i - \mathrm{mean}\left(r_1, \ \ldots, r_G\right)}{\mathrm{std}\left(r_1, \ \ldots, r_G\right)}$$

ρ is the new-to-old policy ratio; clip and KL bound the update.

ONE QUESTION · G = 4 ANSWERS

reward / advantage / update

A · r = 0.20	−1.5 ↓
B · r = 0.90	**+1.1 ↑**
C · r = 0.50	−0.4 ↓
D · r = 0.80	**+0.7 ↑**

ADVANTAGE SETS THE DIRECTION

Â > 0 raise · **Â < 0** lower · **Â ≈ 0** little effect

No critic; the group mean is the baseline.

> Within one question, let the **better answer win**.

Math and code: cheap feedback, transferable skills

Closed environments, traceable errors, clean data.

01 · CLOSED ENVIRONMENT

Axioms · runtime

Math uses definitions and inference rules; code uses language specs and runtime.

02 · TRACEABLE ERRORS

Locate the step

Proofs are checked step by step; tests and stack traces locate code errors.

03 · CLEAN DATA

Parse and filter

Formal objects and code are structured, easy to generate and verify at scale.

MATH TRAINING

State assumptions · keep symbols and logic consistent · build and check multi-step arguments

PROGRAMMING TRAINING

Track variables and state · respect types and constraints · decompose and debug

> Cheap feedback enables scale; shared skills enable transfer.

Four tasks, different feedback conditions

Code, math, product design, trading: how do goals, environments and feedback fit?

01 · CODE

Executable checks

- Target: specified behavior
- Feedback: compiler, tests, errors
- Trials: run and revise locally

02 · MATH

Explicit assumptions & goals

- Target: solve or prove under assumptions
- Feedback: answers, proof checks, counterexamples
- Trials: generate and verify candidates

03 · PRODUCT DESIGN

Long real-world feedback loops

- Target: user value and differentiation
- Feedback: slower, plural, empirical
- Judgment: distinctive, sometimes non-consensus

04 · TRADING

Clear reward, changing markets

- Target: profit, risk and cost
- Environment: incomplete info, shifting markets
- Validation: backtests versus future returns

> Checkable goals turn more compute into **systematic search and candidate filtering**.

Summary: environment, feedback, updates

Well-defined tasks let a model predict, receive checks and iterate in an environment.

Problem + environment → Next-token prediction → Check + trace → Define loss → Backpropagate ↺

01 · ENVIRONMENT

Clear rules & goal

The model can act, try and produce outputs.

02 · FEEDBACK

Evaluate & trace

Check outcomes and locate failing steps.

03 · UPDATE

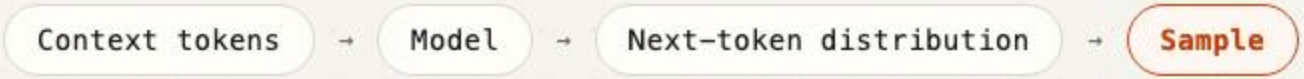
Loss-driven iteration

Backpropagate, update parameters, improve.

> The environment supplies feedback; **loss turns it into parameter updates**.

The model only predicts the next token

Memory, computation, action and goals live outside that action.



01 · NO SAVED STATE

The client resends the history

02 · COMPUTATION IS RECALL

Like a times table: memory, not an algorithm

03 · NO DIRECT ACTION

Tools execute outside

04 · NO PERSISTENT GOAL

Plans are generated text

Four limits → three system capabilities: **state, computation, reliability**; probabilistic output needs verification, stopping, rollback.

> The model gives only the next token; **everything else comes from the system**.

Put the model back in the system



01	**LLM**	A probability generator: tokens in, a distribution over the next token out
02	**Prompt**	This call's conditions: system / user / few-shot
03	**Context**	Every token visible to this call; the window is finite
04	**Memory**	State beyond the window: write / retrieve / compress / forget
05	**Agent loop**	Observe → act → observe; state lives outside
06	**Harness**	Tools, sandbox, permissions, verification, logs

> Memory owns state, tool calls compute, harness owns bounds: together they form an **agent system**.

The inflation of the context window

Context is every token the model can see in this call; the window is its limit.



COST

Billed per token; long prefills are slower

LATENCY

Prefill and attention costs rise

DILUTION

Middle content gets ignored; signal drowns in noise

> A longer window raises the ceiling; relevance decides what lands.

Context and memory: a finite window, used well

The window is a budget, not a warehouse: place the right things, then decide what stays.

01 · FINITE WINDOW

4k → 1M, still lossy

Needle in a haystack: the middle is diluted first.



Capacity ≠ effective use.

02 · PLACEMENT

Workbench, not warehouse

- system · rules
- history · recent turns
- retrieval · relevant docs
- tools · returned results
- current · this question

Too long→retrieve; too scattered→structure; overflow→summarize or sub-agents.

03 · RETENTION

Signal, not logs

Signal
tasks · constraints · decisions · facts

Noise
chat · repetition · stale · raw logs

WRITE
decisions and facts only

RETRIEVE
for this task

COMPRESS
summarize, merge

FORGET
delete stale

> Capacity ≠ effective use; Context decides **what goes in**, Memory decides **what stays**.

Calculation, or recall?

The model predicts the next token; it does not execute multiplication.

13 × 15

$$
\begin{aligned}
&13 \times 15 \\
&= 13 \times 5 + 13 \times 10 \\
&= 65 + 130 \\
&= 195
\end{aligned}
$$

> 3 × 5 = 15 is recall; decomposition and addition are still token prediction.

01 · TIMES TABLE

3 × 5 = 15 is memorized, not computed.

02 · DECOMPOSITION

13 × 15 can be broken into known pieces; decomposing is not arithmetic.

03 · SAMPLING

When data is sparse, the distribution is flat; sampling can pick a wrong answer.

> Deterministic problems need deterministic results; **exact computation belongs to a program**.

Tool calling: let a program do the arithmetic

```
tool schema
{
  "name": "run_python",
  "args": {"code": "str"},
  "returns": "str",
  "limits": ["sandbox", "timeout"]
}

one call
call  run_python("print(2 * 4**1.5)")
obs   "16.0"

// T = 2 × 4**1.5 = 16
```

01 DEFINE

Name, arguments, return, boundary; the description is context too.

02 CALL

The model emits a tool_call; it proposes, does not execute.

03 EXECUTE

The runtime validates arguments and runs in a sandbox; timeouts and permissions are system concerns.

04 OBSERVE

stdout / stderr return to the context; the model decides to continue or stop.

> Tool output is still **untrusted input**: validate arguments, limit permissions, set timeouts, keep logs.

> The model proposes, the program executes; **repeated work becomes a function, not tokens**.

Harness: bound the tools, recover the run

Tools execute in an environment; probabilistic output will eventually fail on long runs.

A controlled loop

Propose → Validate → Execute → Observe → Verify → Decide

TWO PROBLEMS

Tools must not cross bounds: no deleting files, no public uploads.

Long runs must roll back: checkpoints, snapshots, hard stops.

It does not add intelligence; it makes errors **visible, stoppable, reproducible**.

01 TOOL REGISTRATION & VALIDATION
Check schemas and types; block illegal proposals immediately

02 SANDBOX & PERMISSIONS
Separate read and write; restrict permissions, network and directory scope before execution

03 TIMEOUTS & RETRIES
Prevent infinite loops; enforce timeout limits and retry caps

04 RESULT VERIFICATION
Check correctness via independent assertions; never trust self-claims

05 ROLLBACK & STOPPING
Checkpoints, snapshots, idempotency; force a stop on exception

06 LOGS & REPLAY
Record every call and response; ensure full trajectory auditability

> The model provides probability; the harness provides **bounds and a way back**.

Skill: compile the loop into a reusable capability

The loop produces experience; a skill turns it into a loadable procedure.

01 · RUN

Loop

Observe → Act ↺

State lives outside; runs for hours or days.

02 · EXTRACT

Trajectory

Success | Failure | Repair

Checkpoint

Extract repeatable steps; remove the accidental.

03 · SOLIDIFY

Skill

```
SKILL.md
scripts/
references/
```

Load on demand; reuse next time.

Skill package: SKILL.md trigger and steps · scripts deterministic tools · references domain docs · assets templates

> Loop is runtime; skill is compiled experience; memory is state; harness is boundary.

Acceleration: three steps

Tools → reasoning → scale; each jump is generator + verifier + search.

2023–24
Tools: search + verification

2023.12
FunSearch
LLM + evaluator + evolutionary search

2024.07
AlphaProof
Lean + RL; IMO silver 28/42

2024–25
Reasoning: RL + test-time compute

2024.09
OpenAI o1
RL learns to think; test-time compute

2025.01
· "DeepSeek moment"
DeepSeek-R1
RLVR; reasoning behavior emerges

2025–26
Scale: benchmark to open problems

2025.07
IMO 2025
Gold-level 35/42; natural language, no tools

2026
OpenAI
Unit-distance counterexample; NS claims; 372 families

2024.07 **28/42** silver → 2025.07 **35/42** gold-level 12 months, +7 points · role: assistant → candidate generator

> Every jump is **generator + verifier + search**.

How they describe it

> Public material covers part of the method; the full training recipe stays private. Note the asymmetry: some results are easy to verify, hard to prove. Quotes below are **public statements**, excerpted or paraphrased.

DATE	COMPANY · SYSTEM / WORK	PUBLIC STATEMENT	READING
2023.12	**Google DeepMind · FunSearch**	"pairing a pre-trained LLM ... with an automated evaluator"	search
2024.01	**Google DeepMind · AlphaGeometry**	"100 million synthetic theorems and proofs"	synthetic data
2024.07	**Google DeepMind · AlphaProof**	autoformalization → Lean; RL + test-time RL	formalization
2024.09	**OpenAI · o1**	"spend more time thinking through problems"	reasoning
2025.01	**DeepSeek · R1**	"directly apply RL ... rule-based rewards"	RLVR
2025.07	**OpenAI · IMO 2025**	"natural language ... no tools"	end to end
2026.09–10	**OpenAI · 2026 report**	NS breakdown claim; 372 families, 722 manuscripts at launch	open problems

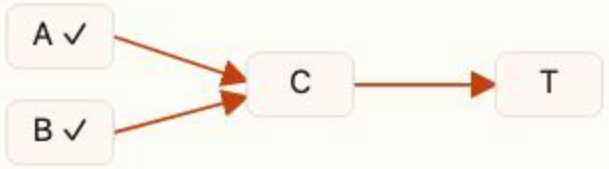
> The keywords change; the underlying move stays: **propose, verify, search again**.

Generate → verify → search: how proof graphs evolve

The mechanism is a hypothesis; the bottlenecks are real.

Generate · LLM → Verify · Lean / tests / numerics → **Search · best-of-N / tree / evolution** ↺

DAG · PROOF GRAPH



Nodes: lemmas / subgoals; edges: dependencies; proved lemmas are reused. Agent expands, Lean checks.

MCTS · SEARCH



Select → expand → evaluate → backpropagate; concentrate budget on promising branches.

System bottleneck: decomposition · shared state · concurrency · budget | **Model bottleneck:** key step · lemmas · strict proof

> Lean guards formal logic; **system and model together determine how far we go**.

OpenAI: research mathematics at batch scale

From a single demonstration to a collection for review and follow-on research.

722
Manuscripts at launch

372
Related result families

≈4,000
Problems attempted

WHAT WAS RELEASED

Manuscripts and proof artifacts
Lean formalizations for some results
10 reasoning summaries; compute data

HOW I READ THE NUMBERS

One family: several papers
3 Pro hours: compute equivalent
Truth, novelty, significance: review

> Candidate results arrive in batches; **verification and understanding matter**.

Navier–Stokes: a long-running multiagent effort

OpenAI reports finite-time breakdown in smooth, forced 3D incompressible flow.

SCOPE OF THE REPORTED RESULT

Positive viscosity; smooth data & force

Whole space $\mathbb{R}^3$ and periodic flow

Finite-time breakdown; Clay C / D

A breakdown construction; unforced NS is separate.

≈10^4	88 h	+17 h	130B
Agents	Find argument	Formalize	Output tokens

Explore in groups → consolidate findings

Share → search further → Lean checks

10k agents → scheduling, checkpoints, retries, stopping

> Check the logic; **also audit the problem–statement–paper correspondence**.

Theorem-cracking system: LLM as CPU

Not just a mathematics problem; a machine for throughput and stability.

WORKLOAD	**Theorem · proof graph**	what to solve
CPU	**LLM**	propose next
SCHEDULER	**Loop + Harness**	concurrency · retries · checkpoints · stop
VERIFIER	**Lean / tools**	correctness · compute
STATE	**Context + Memory**	graph · logs · budget

01 · NOT ONLY MATHEMATICS

Decomposition, scheduling, state, verification

02 · THROUGHPUT

Concurrency, caching, reclamation, cost

03 · STABILITY

Long runs, recovery, auditability

04 · SCALABLE

Swap problems, not the system

> The model is the CPU; **theorem cracking needs the whole machine**.

Failure probability: long chains amplify 1%

The proof graph is read and written constantly; errors compound along dependencies.

END-TO-END SUCCESS · ILLUSTRATIVE

Per step	100	1,000	10,000
99%	37%	0.0043%	≈0
99.9%	90%	37%	0.0045%
99% + 3 retries	≈100%	99.9%	99%

Assumes independent failures; real errors correlate, and retries are not free.

DETECT ERRORS

Lean, tests, assertions first

RETRY SAFELY

Backoff, caps, no duplicate effects

ROLL BACK STATE

Checkpoints, idempotency, recovery

> The system does not prevent every error; **it makes errors non-fatal**.

Systems engineering: six interfaces

Each layer can work alone; the whole machine still may not be reliable.

01 · STATE CONSISTENCY
Concurrent graph edits
Versions, locks, conflict merge

02 · SCHEDULING & BUDGET
Who runs, how long
Concurrency, preemption, cost

03 · EXECUTION FAILURE
No duplicate side effects
Idempotency, backoff, caps

04 · VERIFICATION INTERFACE
Natural language ↔ Lean
Semantic gaps and assumptions

05 · OBSERVABILITY
Which layer failed?
Execution · program · tool · search path

06 · HUMAN BOUNDARY
When to stop, who owns
Automate execution, gate judgment

Low-hanging fruit: hard to prove, easy to disprove; one counterexample settles it · **Real systems problem**: reproducible, scalable, maintainable

> Low-hanging fruit proves direction; **the hard part is the whole machine**.

From the next token to the next theorem

`token` → `probability` → `candidate` → `verification` → **`theorem`**

Probability gives breadth

Verification gives correctness

People set the direction

> The next theorem still begins with the **next token**.

Thank you